

OptiCrop – AI-Based Crop Recommendation System

Project Description:

OptiCrop is a Machine Learning-based web application designed to recommend the most suitable crop based on soil nutrients and environmental conditions. The system helps farmers, agricultural officers, and researchers make informed crop selection decisions, improving productivity and supporting sustainable farming practices.

The application allows users to enter soil and environmental parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall. Using a trained Scikit-learn Machine Learning model, the system analyzes these inputs and instantly predicts the most suitable crop for cultivation.

Built using Python, Flask, HTML, CSS, and Scikit-learn, OptiCrop provides a modern, responsive, and user-friendly interface with an attractive agriculture-themed design. The trained Machine Learning model is integrated with the Flask backend to deliver fast and accurate crop recommendations. The project follows clean coding practices, is managed through GitHub, and is deployed on Render, making it scalable, maintainable, and easy to enhance with future features.

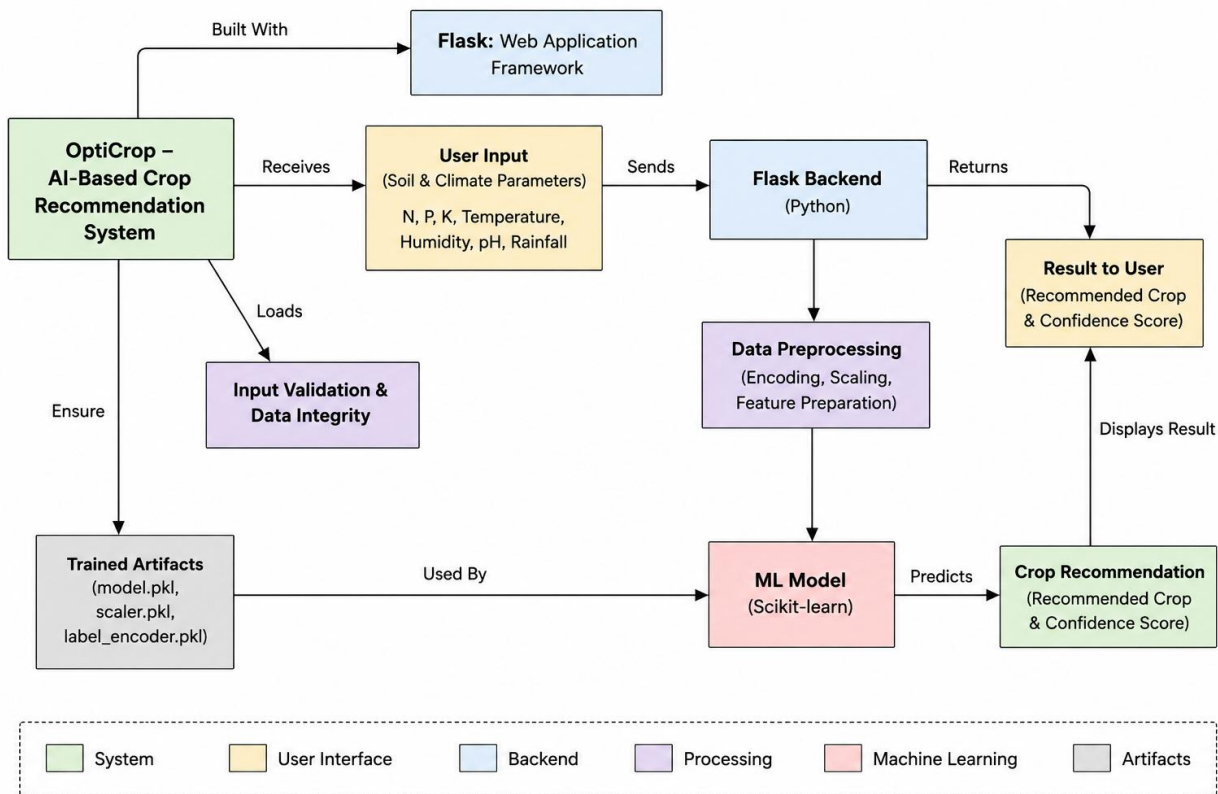
Scenarios

Scenario 1: A farmer wants to cultivate crops on a newly prepared field but is unsure which crop is best suited for the current soil and climate conditions. By entering the soil nutrient values (Nitrogen, Phosphorus, and Potassium), temperature, humidity, pH, and rainfall into the OptiCrop application, the system analyzes the data and instantly recommends the most suitable crop, helping the farmer make an informed cultivation decision.

Scenario 2: An agricultural extension officer assists multiple farmers in selecting suitable crops for different regions. Using OptiCrop, the officer enters soil nutrient and environmental data collected from each farm. The application quickly recommends the best crop for each location, enabling better agricultural planning and improving overall crop productivity.

Scenario 3: An agricultural researcher or student uses OptiCrop to study how soil nutrients and environmental conditions influence crop recommendations. By experimenting with different combinations of input values, the system serves as a valuable educational and research tool for precision agriculture, helping users better understand crop suitability under varying conditions.

Technical Architecture:



Description:

OptiCrop follows a three-tier architecture consisting of a Frontend, Backend, and Machine Learning layer. The frontend is developed using HTML and CSS to provide a modern, responsive, and user-friendly interface for collecting soil and environmental parameters from users, including Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall. The Flask (Python) backend receives the user inputs, validates the data, performs preprocessing using the trained StandardScaler, and communicates with the trained Scikit-learn Machine Learning model.

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Scikit-learn Knowledge:** [Scikit-learn Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Pandas & NumPy Knowledge:** [Pandas and NumPy Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Visual Studio Code Documentation](#)

Project Workflow:

Milestone 1: Data Collection and Model Development

- **Activity 1.1:** Collect and analyze the crop recommendation dataset containing soil nutrient values, environmental parameters, and crop labels.
- **Activity 1.2:** Perform data preprocessing, including checking missing values, feature scaling, exploratory data analysis, and dataset preparation.
- **Activity 1.3:** Train and evaluate the Machine Learning model using Scikit-learn for crop recommendation prediction.
- **Activity 1.4:** Save the trained model (model.pkl) and scaler (scaler.pkl) for integration with the web application.

Milestone 2: Backend Development

- **Activity 2.1:** Develop the Flask backend to handle user requests and application routing.
- **Activity 2.2:** Integrate the trained Machine Learning model with the Flask application to generate real-time crop recommendations.

Milestone 3: Application Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for each feature and integrating the Machine Learning prediction model.
- **Activity 3.2:** Implement input validation and preprocessing before passing data to the trained model.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop responsive web pages using HTML and CSS for user interaction.
- **Activity 4.2:** Create Flask templates to collect soil and environmental information and display crop recommendation results.

Milestone 5: Deployment

- **Activity 5.1:** Configure the project environment and install all the required Python dependencies.
- **Activity 5.2:** Deploy the OptiCrop application using Render and verify that the application functions correctly in the hosted environment.

Milestone 6: Conclusion

The OptiCrop project successfully integrates Machine Learning with Flask to provide quick and reliable crop recommendations. The system offers an easy-to-use interface, real-time prediction capability, and a scalable architecture that can be enhanced with additional features such as fertilizer recommendations, weather forecasting, crop disease detection.

Milestone 1: Data Preparation and Model Development

In this milestone, the focus is on preparing the crop recommendation dataset and developing a Machine Learning model capable of recommending the most suitable crop. This involves collecting the dataset, preprocessing the data, selecting an appropriate Machine Learning algorithm, training the model, evaluating its performance, and preparing it for integration with the Flask web application

Activity 1.1: Collect and Prepare the Dataset

Before developing the prediction model, a suitable crop recommendation dataset is required. Follow the steps below to prepare the dataset for model training.

- **Step 1 — Obtain the Dataset:** Download the **Crop Recommendation Dataset** from Kaggle using the following link:
- <https://www.kaggle.com/datasets/chitrakumari25/smart-agricultural-production-optimizing-engine>



Smart_Agricultural Production Optimizing Engine
 Smart Agriculture is being improved by the Artificial Intelligence (AI)

About Dataset

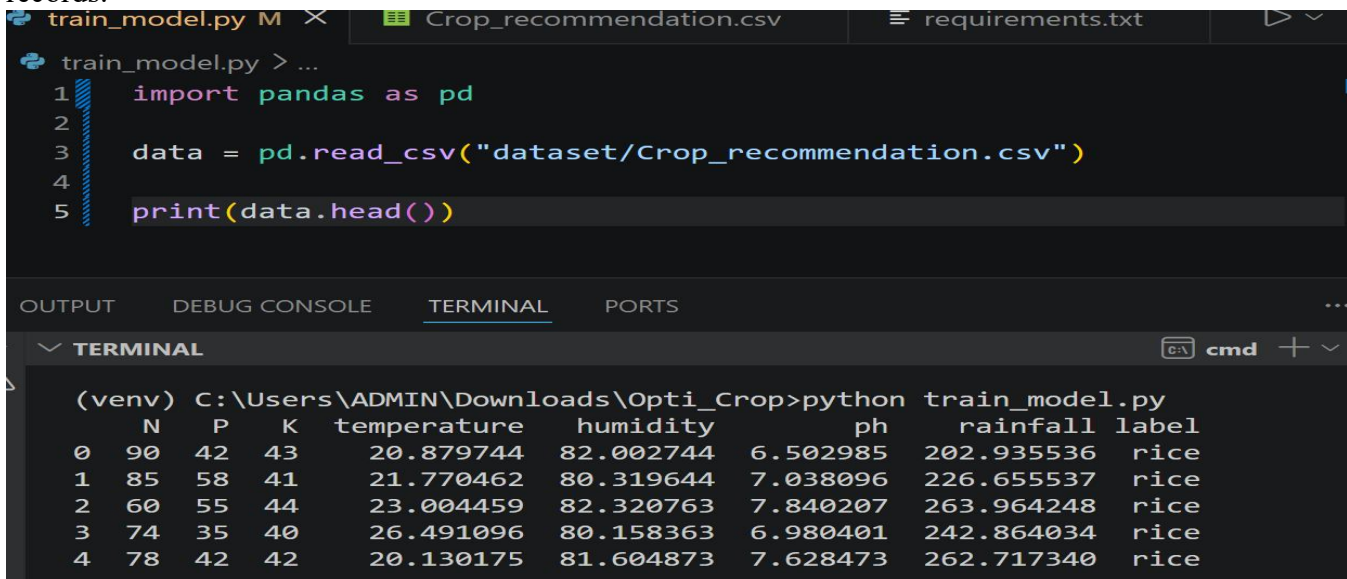
Need For Efficiency In Agriculture- More Than Ever!
 Need For Efficiency In Agriculture- More Than Ever!
 In one of our earlier blog post, Agriculture to AgriTech: Trends, Challenges, and the Path Forward with Digital Technology and Software Solutions, we projected that the agriculture industry would feed an estimated global population of 9.7 billion by 2050. In 2020 alone, a 60% increase is required to feed the population. We talked about macroeconomics, changing consumer preferences, emerging technologies and transforming supply chains as the key drivers for digital transformation in agriculture and how the challenges facing the agriculture industry worldwide could be effectively tackled by following the right approach and leveraging technology to meet the growing demand for food.

Until now, factors such as climate change, population growth and food security concerns have propelled the industry into seeking more innovative approaches to improve crop yield. But the current COVID-19 crisis has further exposed the vulnerability of the agricultural landscape and raised questions about meeting the global food demands sustainably, with adverse factors at play. Again, the answer lies in achieving efficiency: producing more with less, now more than ever.

"The industry will be transformed by data science and artificial intelligence. Farmers will have the tools to get the most from every acre."

Usability 10.00
License ODC Public Domain Dedication ...
Expected update frequency Never
Tags Beginner, Intermediate, Advanced, Agriculture, Clustering

- **Step 2 —Load the Dataset:** Import the Crop Recommendation Dataset into Python using the **Pandas** library and verify that the dataset has been loaded successfully by displaying few records.



```
train_model.py M X Crop_recommendation.csv requirements.txt
```

```
train_model.py > ...
1 import pandas as pd
2
3 data = pd.read_csv("dataset/Crop_recommendation.csv")
4
5 print(data.head())
```

TERMINAL

```
(venv) C:\Users\ADMIN\Downloads\Opti_Crop>python train_model.py
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

- **Step 3 — Explore the Dataset:** Analyze the dataset by checking column names, missing values, data types, crop distribution, and statistical summaries.

```
train_model.py > ...
1  import pandas as pd
2
3  data = pd.read_csv("dataset/Crop_recommendation.csv")
4
5  print(data.head())
6  # Dataset Information
7  print(data.info())
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

▼ **TERMINAL**

```
4  78  42  42    20.130175  81.604873  7.628473  262.717340  rice
<class 'pandas.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0    N                    2200 non-null  int64
1    P                    2200 non-null  int64
2    K                    2200 non-null  int64
3  temperature          2200 non-null  float64
4  humidity              2200 non-null  float64
5    ph                   2200 non-null  float64
6  rainfall              2200 non-null  float64
7  label                 2200 non-null  str
dtypes: float64(4), int64(3), str(1)
memory usage: 137.6 KB
None
```

- **Step 4 — Clean the Dataset:** Handle missing values, remove inconsistencies, and prepare data for model training.

```
train_model.py > ...
34
35  print("\nMissing Values:")
36  print(data.isnull().sum())
37
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

▼ **TERMINAL**

```
dtypes: float64(4), int64(3), str(1)
memory usage: 137.6 KB

Missing Values:
N          0
P          0
K          0
temperature 0
humidity    0
ph          0
rainfall    0
label       0
dtype: int64
```

- **Step 5 — Encode Categorical Data:** Since all input features are numerical, no additional encoding is required for the dataset.
- **Step 6 — Scale Numerical Features:** Standardize the numerical features using StandardScaler before training the Machine Learning model.
- **Important** — Save the trained Machine Learning model (**model.pkl**) and scaler (**scaler.pkl**) in the **model/** folder. These files are used by the Flask application to generate real-time crop recommendations.

Activity 1.2: Select and Train the Machine Learning Model

The following steps were carried out to select the most suitable Machine Learning model for Opti-Crop recommendation.

- **Understand the Project Requirements:** Review the project objectives and identify the soil and environmental parameters required for accurate crop recommendation.
- **Explore Scikit-learn Documentation:** Study suitable Machine Learning algorithms, preprocessing techniques, and evaluation methods for classification tasks.
- **Evaluate Model Performance:** Compare different Machine Learning algorithms using accuracy and overall prediction performance on the crop recommendation dataset.
- **Select the Optimal Model:** Choose the best-performing model, train it on the preprocessed dataset, and save `model.pkl` and `scaler.pkl` for Flask integration.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a diagram showing the frontend, Flask backend, preprocessing module, Machine Learning model, and crop recommendation result.
- **Detail Frontend Functionality:** Explain how users enter N, P, K, temperature, humidity, pH, and rainfall values through the web interface.
- **Outline Backend Responsibilities:** Describe how the Flask backend receives requests, validates inputs, preprocesses data, and sends it to the trained model.
- **Describe Machine Learning Integration:** Explain how the application loads `model.pkl` and `scaler.pkl`, preprocesses inputs, generates the crop recommendation, and returns the result to the frontend

Activity 1.4: Set Up the Development Environment

- **Install Python and pip:** Ensure Python 3.8 is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv venv  
D:\projects>venv\Scripts\activate  
(venv) D:\projects> pip install -r requirements.txt
```

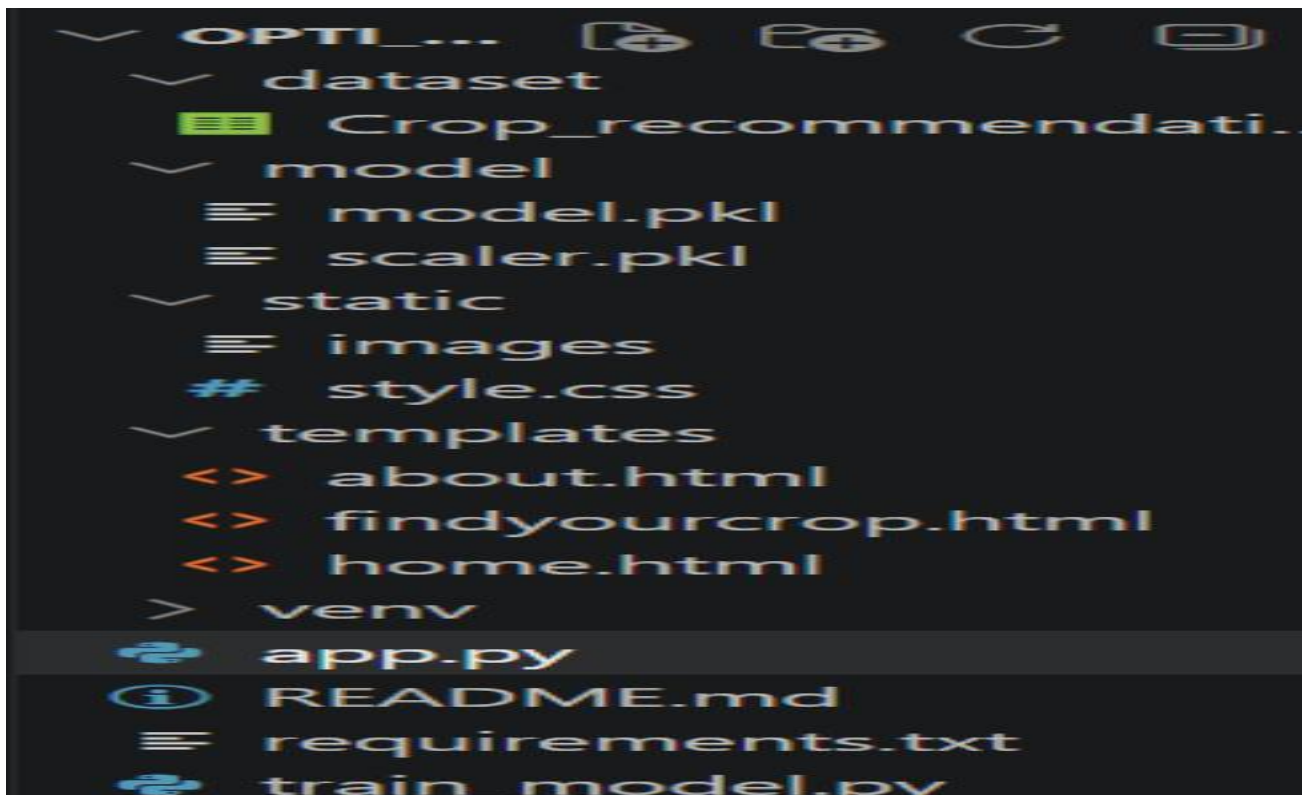
- **Install Flask:** Use pip to install Flask:

```
(venv) D:\projects> pip install Flask
```

- **Install Required Libraries:** Install the required libraries for Machine Learning and data processing:

```
(venv) D:\projects> pip install pandas numpy scikit-learn joblib matplotlib seaborn
```

- **Configure the Project:** Ensure all the required project files, including the dataset (**Crop_recommendation.csv**), trained model (**model.pkl**), and scaler (**scaler.pkl**), are placed in their respective folders before running the application.
- **Set Up Application Structure:** Create the project directory structure including folders for dataset, model, templates, static (CSS & Images), and the main application files (app.py, train_model.py, and requirements.txt)



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Crop Recommendation Features

Implement the core functionalities required for the OptiCrop application to provide accurate crop recommendations through a user-friendly web interface.

- **Crop Recommendation:** Predict the most suitable crop based on soil nutrients and environmental conditions using the trained Machine Learning model.
- **Data Preprocessing:** Process user inputs by applying the same preprocessing techniques used during model training, including feature scaling before generating predictions.
- **Prediction Result:** Display the recommended crop instantly through the Flask web application, providing users with a simple and understandable result.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in **app.py** for the Home page, About page, Find Your Crop page, and crop prediction endpoint.

Process User Input:

- Create a form in **findyourcrop.html** for users to enter soil and environmental parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall.
- Use Flask's **request.form** to retrieve the submitted user data from the frontend.
- Validate the input values to ensure all mandatory fields are provided before processing the crop recommendation request.

Integrate The Machine Learning Model:

- Load the trained Machine Learning model (**model.pkl**) and feature scaler (**scaler.pkl**) into the Flask application.
- Apply the same preprocessing and feature scaling techniques used during model training to the user input.
- Generate the crop recommendation using the trained model and return the prediction result to the frontend for display.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the **app.py** file, which serves as the backbone of the **OptiCrop** application. In this milestone, the Flask routes are developed to handle user requests, process soil and environmental parameters, integrate the trained Machine Learning model, and return crop recommendations through the web interface.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish routes for **OptiCrop's** endpoints, each serving a distinct purpose:
 - **/** — Home page route that renders the main home.html interface.

```
@app.route("/")
def home():
    return render_template("home.html")
```

- **/about** — Route that displays information about the OptiCrop project.

```
@app.route("/about")
def about():
    return render_template("about.html")
```

- **/findyourcrop** — Route that displays the crop recommendation form.

```
@app.route("/findyourcrop")
def findyourcrop():
    return render_template("findyourcrop.html")
```

- **/predict** — POST endpoint that accepts soil and environmental parameters, preprocesses the input, invokes the trained Machine Learning model, and returns the crop recommendation.

```
@app.route("/predict", methods=["POST"])
def predict():

    values = [float(x) for x in request.form.values()]

    values = np.array(values).reshape(1, -1)

    values = scaler.transform(values)

    prediction = model.predict(values)
```

```
return render_template(  
    "findyourcrop.html",  
    prediction_text=f"Recommended Crop: {prediction[0]}"  
)
```

Set Up Route Handlers for Each Feature:

- For the **/predict** route, implement logic that reads the soil and environmental parameters submitted through the HTML form using `request.form`.
- Apply input validation by converting the submitted values into the appropriate numerical format and ensuring that all mandatory input fields are provided before processing the crop recommendation request.
- Route form submissions from the frontend to the **/predict** endpoint, where the user input is preprocessed using the trained scaler, passed to the Machine Learning model for prediction, and the recommended crop is displayed on the result page.

Define Machine Learning Prediction Helpers:

- **Load the Trained Model:** Load the trained Machine Learning model (model.pkl) and feature scaler (scaler.pkl) at application startup to perform crop recommendations.

```
model = joblib.load("model/model.pkl")
scaler = joblib.load("model/scaler.pkl")
```

- **Retrieve User Input:** Collect the soil and environmental parameters submitted through the prediction form using Flask's request.form and convert them into numerical values.

```
values = [float(x) for x in
request.form.values()]
```

- **Preprocess Input Data:** Convert the user input into a NumPy array, reshape it into the required format, and apply the trained StandardScaler before prediction.

```
values = np.array(values).reshape(1, -1)

values = scaler.transform(values)
```

- **Generate Crop Recommendation:** Pass the preprocessed data to the trained Machine Learning model and display the recommended crop to the user.

```
prediction = model.predict(values)

return render_template(
    "findyourcrop.html",
    prediction_text=f"Recommended Crop:
{prediction[0]}"
)
```

Integrate the Machine Learning Prediction in Each Function:

- Load the trained Machine Learning model (model.pkl) and feature scaler (scaler.pkl) when the Flask application starts.
- Retrieve the soil and environmental parameters from the prediction form using request.form, convert the values into numerical format, and prepare them for prediction.
- Apply the trained StandardScaler (scaler.pkl) to preprocess the user input before passing it to the Machine Learning model for crop recommendation.

```
data = scaler.transform(data)
prediction = model.predict(data)
```

- Generate the crop recommendation and display the predicted crop on the **Find Your Crop** page.

```
• if prediction[0] == 1:
•     result = "Loan Approved ☒ "
• else:
•     result = "Loan Rejected ☐ "
•
• return render_template("submit.html", prediction=result)
```

Application Routes:

- **Home route:** / → Renders home.html
- **About Route:** /about → Displays information about the OptiCrop project (about.html)
- **Find Your Crop Route:** /findyourcrop → Displays the Crop Recommendation page (findyourcrop.html), where users can enter soil nutrient values (N, P, K) and environmental parameters such as temperature, humidity, pH, and rainfall to obtain the most suitable crop recommendation.
- **Prediction Route (POST):** /predict → Accepts soil and environmental parameters, preprocesses the data using the trained scaler, generates the crop recommendation using the Machine Learning model, and displays the recommended crop in findyourcrop.html.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a simple, responsive, and user-friendly interface for the OptiCrop application. The frontend is designed using HTML and CSS to provide users with an intuitive environment for entering soil and environmental parameters and viewing the crop recommendation generated by the Machine Learning model.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop separate HTML pages (`home.html`, `findyourcrop.html`, and `about.html`) to provide navigation, crop recommendation input, and project information.
- Use semantic HTML elements such as headings, forms, input fields, buttons, cards, and containers to organize the user interface and improve readability..
- Create an attractive Home page introducing the OptiCrop application and provide a button to navigate to the Find Your Crop page..

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (`static/style.css`) to define the application's visual appearance, including colors, typography, spacing, buttons, cards, forms, navigation bar, and animated background.
- Use responsive layouts and consistent styling to ensure the application is easy to use across different screen sizes.
- Apply hover effects, animations, and styling to buttons, cards, navigation links, and form elements to improve user experience.

Create Separate UI Components:

- **Home Page:** Displays the OptiCrop title, project description, AI-powered farming features, statistics, and navigation menu.
- **Find Your Crop Page:** Collects soil nutrient values such as Nitrogen (N), Phosphorus (P), Potassium (K), and environmental parameters including Temperature, Humidity, pH, and Rainfall using HTML input fields.
- **Prediction Result:** Displays the recommended crop generated by the Machine Learning model on the same Find Your Crop page, allowing users to modify the input values and generate another recommendation.

Activity 4.2: Handling User Input Through Flask

Handle Form Submission:

- Create an HTML form in findyourcrop.html that submits soil and environmental parameters to the Flask backend using the POST method.
- Validate that all required input fields, including Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall, are completed before submitting the form.

Display Prediction Results:

- Process the submitted form through the /predict route in app.py.
- Pass the crop recommendation from the Flask backend to findyourcrop.html using render_template().
- Display the recommended crop clearly on the Find Your Crop page and allow users to enter new values to generate another crop recommendation.

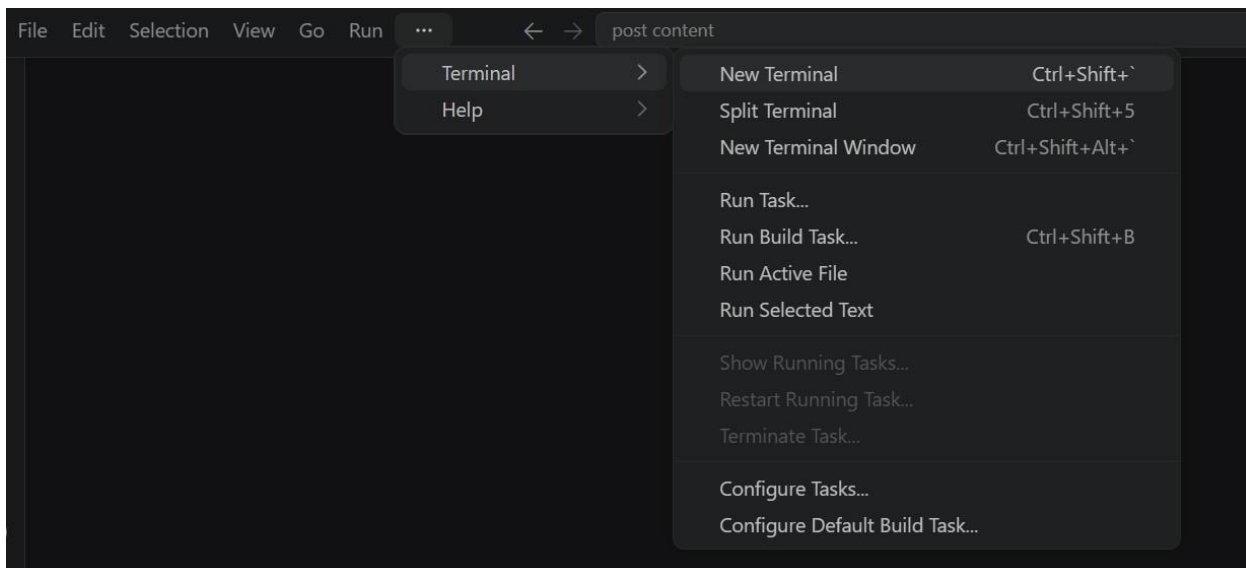
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the **OptiCrop** application locally and on the Render cloud platform to verify its functionality and ensure the Machine Learning model integrates correctly with the Flask web application. This involves setting up the development environment, installing the required dependencies, running the Flask server, and testing the crop recommendation system.

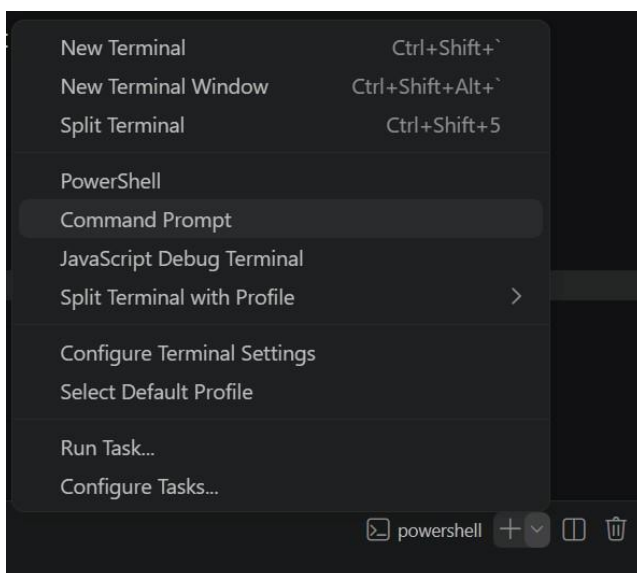
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv venv
D:\projects>venv\Scripts\activate
(venv) D:\projects>pip install -r requirements.txt
```

Install Required Dependencies:

- Install all the required Python libraries listed in requirements.txt, including Flask, Pandas, NumPy, Scikit-learn, Joblib, Matplotlib, and Seaborn.

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(venv) D:\projects>python app.py
```

- Once the server starts successfully, open a web browser and navigate to the <http://127.0.0.1:5000/> to access the application.

```
(venv) C:\Users\ADMIN\Downloads\Opti_Crop>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
```

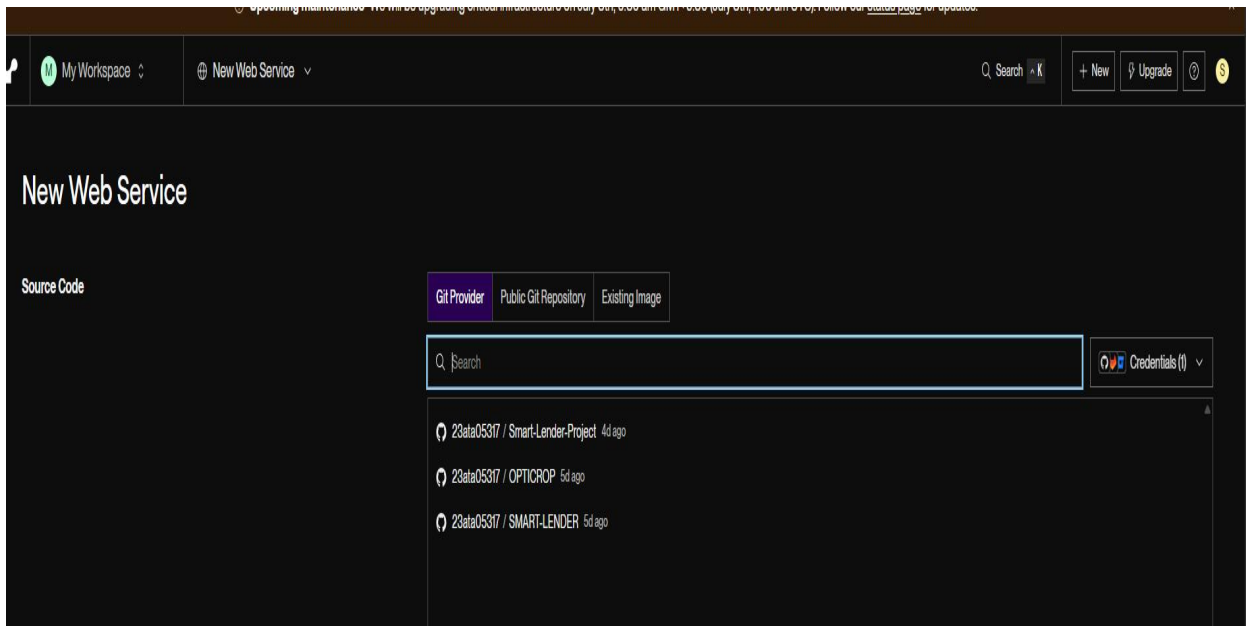
- Interact with the OptiCrop application by entering different soil nutrient values (Nitrogen, Phosphorus, and Potassium) along with environmental parameters (Temperature, Humidity, pH, and Rainfall) to verify that the Machine Learning model correctly processes the input and displays accurate crop recommendations.

Activity 5.3: Public Deployment Using Render

Render is a cloud hosting platform that allows Flask applications to be deployed online, making the OptiCrop application accessible through a secure public URL without requiring users to install the project locally.

Deploy the Application on Render:

- Sign in to the Render dashboard using your GitHub account.
- Click "New +" and select "Web Service".
- Connect your OptiCrop GitHub repository to Render.
- Configure the required build and start commands for the Flask application



Configure the Deployment Settings:

- Set the Build Command as:

```
pip install -r requirements.txt
```

- Set the Start Command as:

```
gunicorn app:app
```

- Click Deploy Web Service to start the deployment process.

Access the Public Application:

- After the deployment is completed successfully, Render generates a public URL for the application.

Live Application URL:

<https://opti-crop.onrender.com>

- Open the URL in any web browser to access the OptiCrop application from anywhere without installing any software.

=====

 YOUR OPTICROP APPLICATION IS LIVE!

=====

Available at your primary URL:

<https://opti-crop.onrender.com>

=====

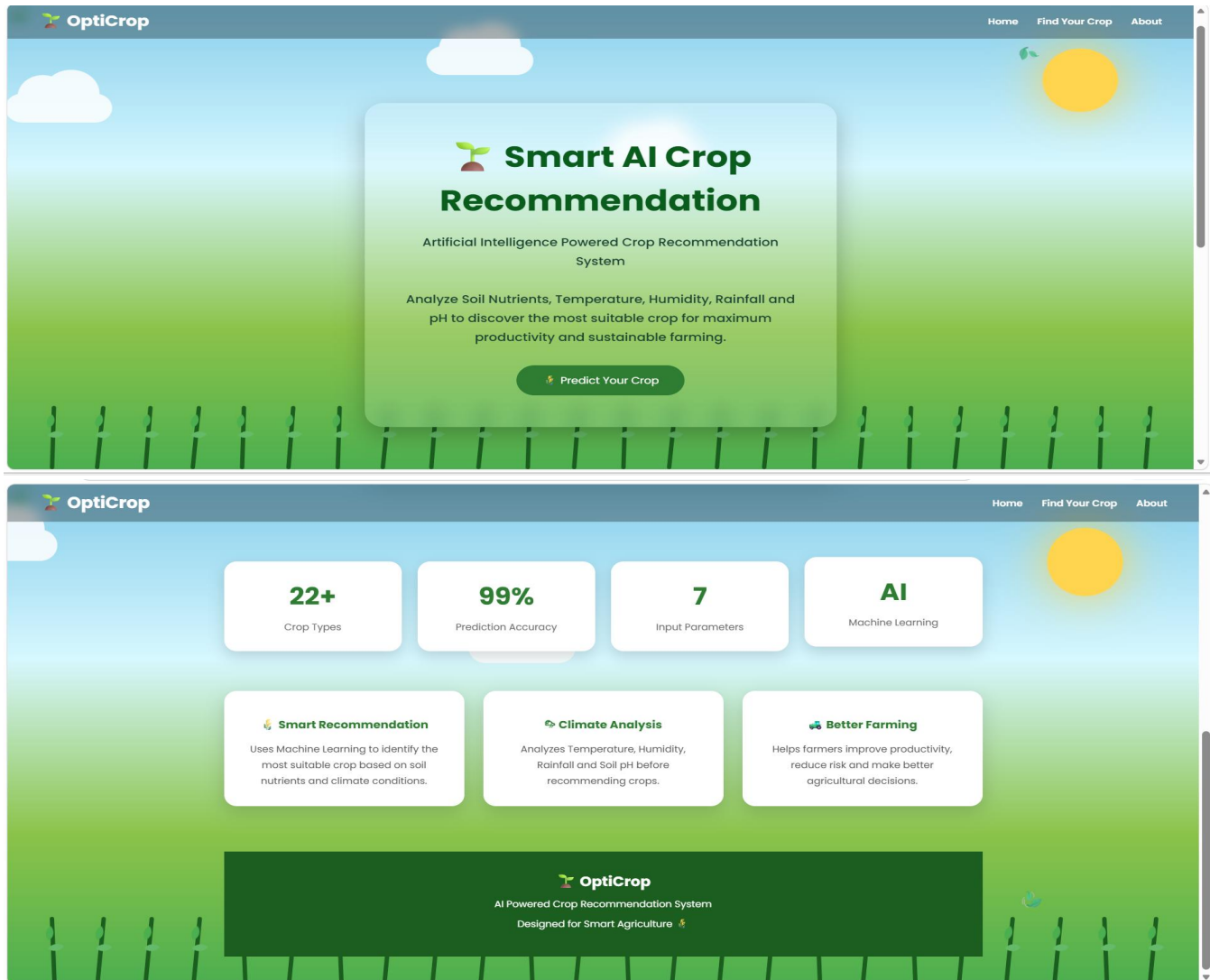
- Share this URL with anyone so they can access the OptiCrop application directly from their browser without installing any software

Important Notes:

- Ensure that all required project files, including the trained Machine Learning model (model.pkl) and feature scaler (scaler.pkl), are included before deploying the application.
- The application is deployed on the Render cloud platform and can be accessed using the public deployment URL.
- Whenever changes are pushed to the connected GitHub repository, Render automatically rebuilds and redeploys the application with the latest updates.
- If the application becomes inactive, Render's free plan may take a short time to restart the service when it receives a new request.
- Verify that the application is running successfully by opening the deployed URL and testing the crop recommendation functionality with different soil nutrient and environmental parameter values.
- Ensure that all required Python dependencies are listed in requirements.txt so that Render can install them successfully during deployment.
- Before deployment, verify that the Flask application runs correctly in the local development environment to avoid deployment errors.

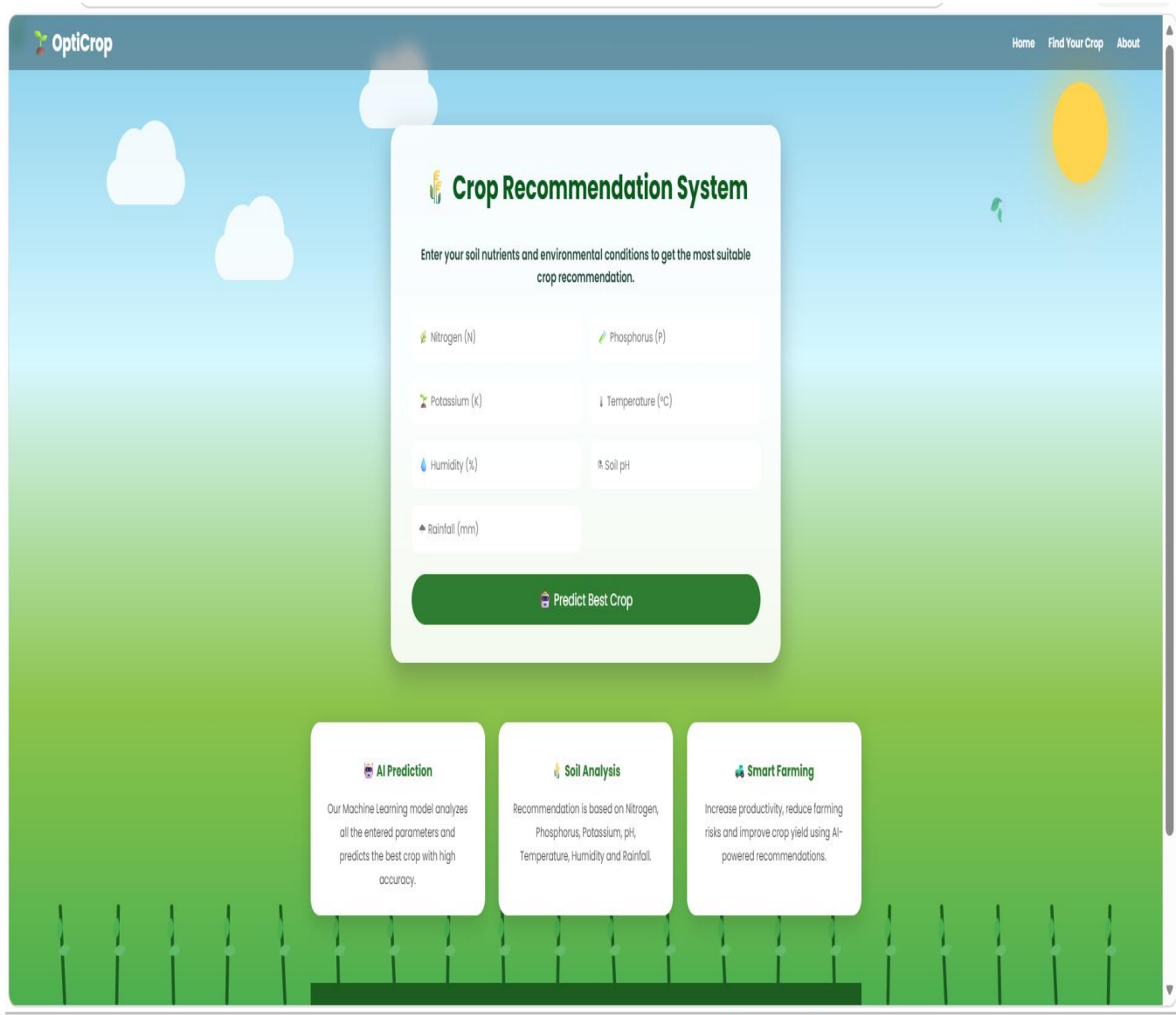
Exploring the Website's Web Pages:

Home Page:



Description: The Home page serves as the landing page of the OptiCrop application. It introduces users to the AI-powered crop recommendation system with an attractive and responsive interface featuring an animated farm background. The page provides a brief overview of the application's purpose, explaining how Machine Learning is used to recommend the most suitable crop based on soil nutrients and environmental conditions

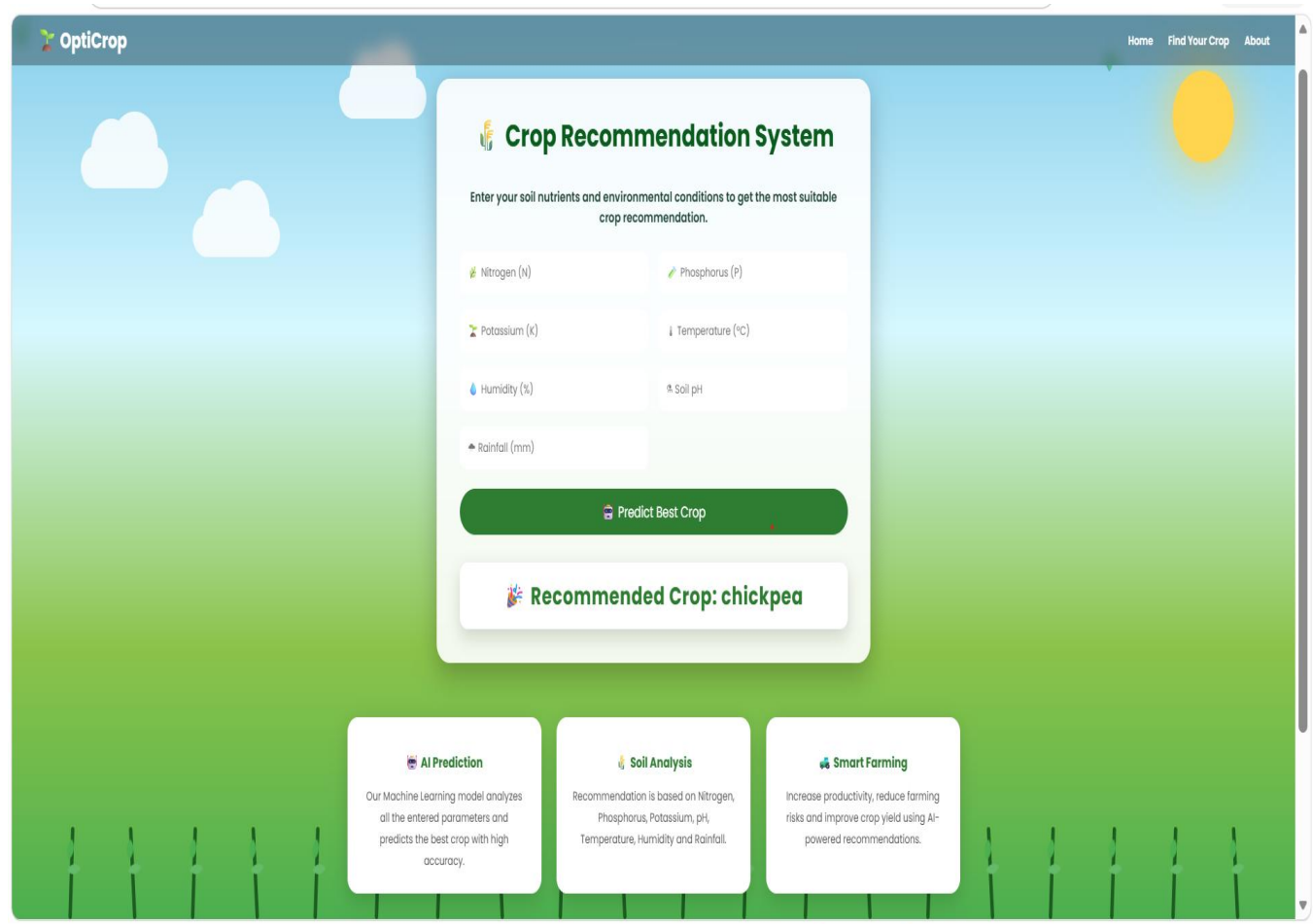
Input Section:



The screenshot shows the 'OptiCrop' web application. The header includes the 'OptiCrop' logo and navigation links for 'Home', 'Find Your Crop', and 'About'. The main content area features a 'Crop Recommendation System' form with the instruction: 'Enter your soil nutrients and environmental conditions to get the most suitable crop recommendation.' The form contains input fields for Nitrogen (N), Phosphorus (P), Potassium (K), Temperature (°C), Humidity (%), Soil pH, and Rainfall (mm). A green 'Predict Best Crop' button is at the bottom of the form. Below the form, there are three informational boxes: 'AI Prediction' (describing the machine learning model), 'Soil Analysis' (listing the parameters used), and 'Smart Farming' (highlighting the benefits of AI-powered recommendations).

Description: The Find Your Crop page provides a simple and user-friendly interface where users can enter the required soil and environmental parameters, including Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, Soil pH, and Rainfall. After entering all the values, users click the "Predict Best Crop" button to submit the information. The input data is sent to the Flask backend, where it is validated, preprocessed using the trained StandardScaler, and passed to the Machine Learning model. The application then analyzes the inputs and instantly displays the most suitable crop recommendation, enabling users to make informed agricultural decisions through an easy-to-use web interface.

Results Section:



The screenshot shows the OptiCrop web application interface. At the top, there's a navigation bar with 'OptiCrop' on the left and 'Home', 'Find Your Crop', and 'About' on the right. The main content area has a light blue sky background with clouds and a sun. A central white box titled 'Crop Recommendation System' contains a form for entering soil nutrients and environmental conditions. The form includes input fields for Nitrogen (N), Phosphorus (P), Potassium (K), Temperature (°C), Humidity (%), Soil pH, and Rainfall (mm). Below the form is a green button labeled 'Predict Best Crop'. Underneath the button, a white box displays the 'Recommended Crop: chickpea'. At the bottom of the interface, there are three white boxes with icons and text: 'AI Prediction' (Our Machine Learning model analyzes all the entered parameters and predicts the best crop with high accuracy), 'Soil Analysis' (Recommendation is based on Nitrogen, Phosphorus, Potassium, pH, Temperature, Humidity and Rainfall), and 'Smart Farming' (Increase productivity, reduce farming risks and improve crop yield using AI-powered recommendations).

Description: The **Result** section displays the crop recommendation generated by the trained Machine Learning model based on the soil nutrients and environmental parameters entered by the user. After the prediction is processed, the application clearly highlights the **recommended crop** on the same page, allowing users to view the result instantly without navigating elsewhere. This immediate feedback enables farmers and agricultural users to quickly identify the most suitable crop for cultivation and make informed farming decisions.

Conclusion:

OptiCrop is a Machine Learning-based web application developed using Flask that simplifies the crop selection process by recommending the most suitable crop based on soil nutrients and environmental conditions. By utilizing a trained Machine Learning model, the application predicts the best crop using input parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall. The project involved dataset collection and preprocessing, model training and evaluation, frontend and backend development, and deployment on the Render cloud platform, demonstrating the practical application of Machine Learning in smart agriculture.

Looking ahead, the OptiCrop application offers several opportunities for enhancement, including integrating real-time weather data through Weather APIs, providing fertilizer recommendations, implementing crop disease prediction, supporting multiple languages, and maintaining prediction history using database integration. By continuously improving the Machine Learning model and enhancing the user interface, OptiCrop has the potential to evolve into a comprehensive intelligent farming assistant that supports farmers in making informed agricultural decisions and promoting sustainable farming practices.